Online Store API - Technical Design (Products Module)

1. Overview

The project involves building and testing the Products APIs of the Online Store. The APIs support CRUD operations, filtering by category, sorting, and limiting results. The design covers both functional requirements and test scenarios for the Products module.

2. Products Module

2.1 API Endpoints

- GET /products: Retrieve all products.
- GET /products/{id}: Retrieve a product by ID.
- GET /products?limit=x: Retrieve a limited number of products.
- GET /products?sort=asc|desc: Retrieve products in a specific order.
- GET /products/categories: Retrieve all product categories.
- GET /products/category/{category}: Retrieve products by category.
- POST /products: Add a new product.
- PUT /products/{id}: Update an existing product.
- PATCH /products/{id}: Partially update a product.
- DELETE /products/{id}: Delete a product.

2.2 Data Model

```
{
  "id": 1,
  "title": "Product Name",
  "price": 100.0,
  "description": "Product Description",
  "category": "Electronics",
  "image": "url-to-image",
  "rating": {
    "rate": 4.5,
    "count": 120
  }
}
```

2.3 Functional Requirements

- Support CRUD operations.
- Enable filtering by category, sorting, and limiting results.
- Handle edge cases like invalid IDs, empty categories, or malformed requests.

3. Testing Design

3.1 Test Cases

- Verify retrieval of all products.
- Verify retrieval of a product by ID.
- Verify retrieval of products with a limit.
- Verify retrieval of products with sorting.
- Verify CRUD operations for products.
- Validate error handling for invalid product IDs.

3.2 Automation Framework

Technology Stack

- Programming Language: Java
- Testing Framework: TestNG or JUnit
- HTTP Client: REST Assured

- Reporting: Allure Reports, Extent Reports
- Data Source: JSON files for test data

Framework Design

- Modular structure with a separate class for the Products module.
- Data-driven testing using JSON or Excel files.
- Reusable utility methods for API requests (GET, POST, PUT, PATCH, DELETE).
- Logging and reporting integration.

4. Non-Functional Requirements

- **Performance:** APIs should respond within 200ms for 90% of requests.
- **Scalability:** APIs should handle 100 concurrent users without degradation.
- **Security:** Implement token-based authentication for protected endpoints.
- **Error Handling:** Return meaningful error messages for invalid requests (e.g., 400 Bad Request, 404 Not Found).

5. Tools and Technologies

- API Development: Node.js, Express.js (or similar backend framework)
- Database: MySQL or MongoDB (depending on use case)
- Testing Tools: REST Assured, Postman, JMeter (for performance testing)
- Version Control: Git
- CI/CD: Jenkins or GitHub Actions for automated testing and deployment

6. Deliverables

1. Functional APIs for all Products endpoints.
2. Automated test cases for all Products scenarios.
3. Documentation covering API specs, test cases, and architecture.